Section 21 : Lambda Expressions

Information

- Title: `Lambda Expressions`
- Authors: `Etienne KOA`

Aperçu de la section

- Qu'est-ce qu'une expression lambda ?
  - Motivation
  - Revue des objets fonctions (foncteurs)
  - Relation entre les lambdas et les objets fonction
- Structure d'une expression lambda
- Types d'expressions lambda
  - Expression lambda apatride
  - Expression lambda avec état
- Lambda et le `STL`

Motivation

- Avant `C++11`
  - Objets de fonction
  - Pointeurs de fonction
  - Nous écrivons souvent de nombreuses fonctions courtes qui contrôlent les algorithmes
  - Ces fonctions courtes seraient encapsulées dans de petites classes pour produire des objets fonctions
  - Souvent, les classes et les fonctions sont très éloignées de l'endroit où elles sont utilisées, ce qui entraîne des problèmes de modification, de maintenance et de test.
  - Le compilateur ne peut pas intégrer efficacement ces fonctions pour plus d'efficacité

Objets de fonction

```
class Multiplier {
private:
   int num{};
public:
   Multiplier(int n) : num {n} {}
   int operator()(int n) const {
      return num* n;
   }
};
```

```
std::vector<int>  vec {1,2,3,4};
Multiplier mult{10};

std::tranform(vec.begin(), vec.end(),vec.begin(),
              mult);

vec now contains {10,20,30,40}
```

```
std::vector<int>  vec {1,2,3,4};

std::tranform(vec.begin(), vec.end(),vec.begin(),
              Multiplier(10));

vec now contains {10,20,30,40}
```

```
template <typename T>
struct Displayer {
    void operator() (const T &data) {
        std::cout << data << " ";
    }
};
```

Objets de fonction générique

```
Displayer<int> d1;
Displayer<std::string> d2;

d1(100);            // d.operator(100);
                    // displays 100
d2("Frank");        // d.operator("Frank");
                    // displays Frank
```

```
std::vector<int> vec1 {1,2,3,4,5};
std::vector<std::string> vec2 {"Larry", "Moe", "Curly"};

std::for_each(vec1.begin(), vec1.end(), Displayer<int>());

std::for_each(vec1.begin(), vec1.end(), d1);

std::for_each(vec2.begin(), vec2.end(), d2);
```

```
std::vector<int> vec1 {1,2,3,4,5};
std::vector<std::string> vec2 {"Larry", "Moe", "Curly"};

std::for_each(vec1.begin(), vec1.end(), Displayer<int>())

std::for_each(vec1.begin(), vec1.end(), d1);

std::for_each(vec2.begin(), vec2.end(), d2);
```

En utilisant une expression lambda

```
std::vector<int> vec1 {1,2,3,4,5};
std::vector<std::string> vec2 {"Larry", "Moe", "Curly"};


std::for_each(vec1.begin(), vec1.end(),
              [](int x) { std::cout << x << " ";};

std::for_each(vec2.begin(), vec2.end(),
              [](std::string s) { std::cout << s << " ";};
```

```
[] () -> return_type specifiers { };
```

Capture List
Defines the start of the lambda

Structure d'une expression Lambda

La structure d'une expression lambda

```
[] () -> return_type specifiers { };
```

Parameter List
Comma separated list of parameters

```
[] () -> return_type specifiers { };
```

Return type
Can be omitted and the compiler will try to deduce it!

```
[] () -> return_type specifiers { };
```

Optional specifiers
mutable and constexpr

```
[] () -> return_type specifiers { };
```

Body
Your code!

```
[] () { std::cout << "Hi";};
```

Une simple expression Lambda

```
[] () { std::cout << "Hi";}();

// Displays Hi
```

```
[] (int x) { std::cout << x;};
```

Paramètres des expressions lambda

```
[] (int x, int y) {std::cout << x + y;};
```

Attribution d'une expression lambda à une variable

```
auto l = [] () { std::cout << "Hi";};

l();    // Displays Hi
```

```
auto l = [] (int x) { std::cout << x;};

l(10);  // displays 10
l(100); // displays 100
```

Renvoi d'une valeur à partir d'une expression lambda

```
auto l = [] (int x, int y) -> int { return x + y;};

or

auto l = [] (int x, int y) { return x + y;};

std::cout << l(2,3);      // displays 5
std::cout << l(10,20);    // display 30
```

Figure 1: lambda

Expression Lambda sans état

```
[] () -> return_type specifiers { };
```

Capture List
Defines the context in which the lambda executes

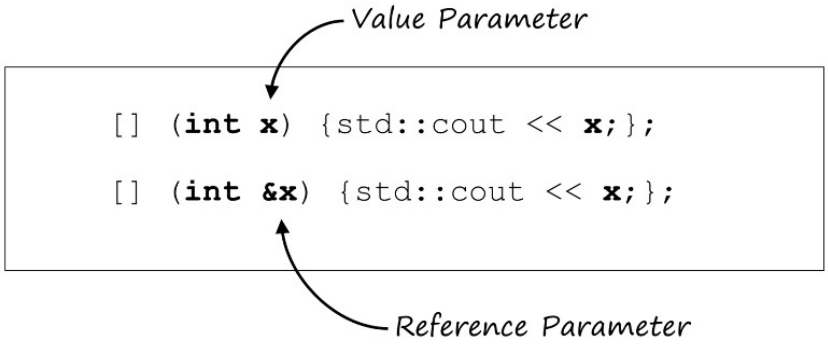
Figure 2: lambda

Expressions lambda apatrides simples

```
[] () {std::cout << "Hi";}();          // Displays Hi

int x {10};

[] (int x) {std::cout << x;}(100);     // Displays 100
```

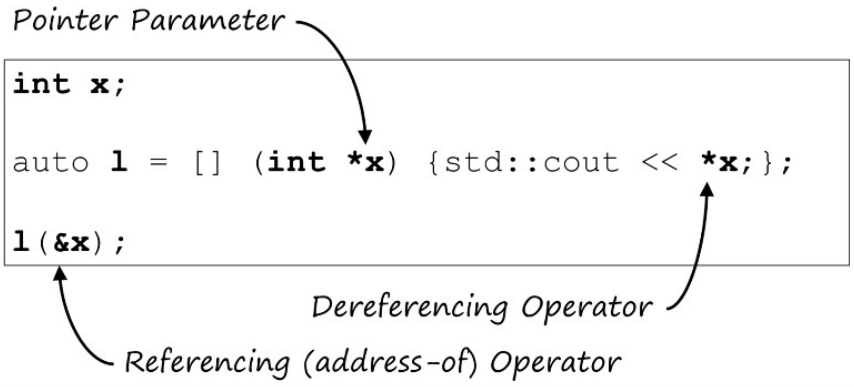
```
const int n {3};
int nums[n] {10,20,30};

auto sum = [] (int nums[], int n) {
     int sum {0};
     for (int i = 0; i < n; i++)
          sum += nums[i];
     return sum;
};

std::cout << sum(nums, 3);          // Displays 60
```

Utilisation de valeurs et de références comme paramètres lambda



```
int test_score1 {88};
int test_score2 {75};

auto bonus = [] (int &score1, int &score2, int bonus_points) {
       score1 += bonus_points;
       score2 += bonus_points;
};

bonus(test_score1, test_score2, 5);

std::cout << "test_score1: " << test_score1 << std::endl;   // Displays 93
std::cout << "test_score2: " << test_score2 << std::endl;   // Displays 80
```

Utilisation de pointeurs comme expression lambda




```
int test_score1 {88};
int test_score2 {75};

auto bonus = [] (int *score1, int *score2, int bonus_points) {
       *score1 += bonus_points;
       *score2 += bonus_points;
};

bonus(&test_score1, &test_score2, 5);

std::cout << "test_score1: " << test_score1 << std::endl;   // Displays 93
std::cout << "test_score2: " << test_score2 << std::endl;   // Displays 80
```

Utilisation de « tableaux » et de « vecteurs » comme paramètres de référence lambda

Utilisation de auto comme spécificateurs de type de paramètre lambda

```
std::vector<int> test_scores {93,88,75,68,65};

auto bonus = [] (std::vector<int> &scores, int bonus_points){
       for (int &score: scores)
              score += bonus_points;
};

bonus(test_scores, 5);

std::cout << "test_scores:"<< std::endl;
std::cout << test_scores[0] << std::endl;      // Displays 98
std::cout << test_scores[1] << std::endl;      // Displays 93
std::cout << test_scores[2] << std::endl;      // Displays 80
std::cout << test_scores[3] << std::endl;      // Displays 73
std::cout << test_scores[4] << std::endl;      // Displays 70
```

Figure 3: lambda

```
int num1 {10};
float num2 {20.5};

auto l = [] (      x) {std::cout << x;};

l(num1);
l(num2);
```

```
int num1 {10};
float num2 {20.5};

auto l = [] (auto x) {std::cout << x;};

l(num1);
l(num2);
```

```
std::vector<int> test_scores1 {93,88,75,68,65};
std::vector<float> test_scores2 {88.5,85.5,75.5,68.5,65.5};

auto bonus = [] (auto &scores, int bonus_points){
      for (auto &score: scores)
            score += bonus_points;
};

bonus(test_scores1, 5);        // Valid
bonus(test_scores2, 5);        // Valid
```

Utilisation de l'expression lambda comme paramètres de fonction

```
#include <functional>                          // for std::function

void foo(std::function<void(int)> l) {         // C++14
      l(10);
}
                Return Type
or                          Parameter Type

void foo(void (*l)(int)) {                     // C++14
      l(10);
}

or

void foo(auto l) {                             // C++20
      l(10);
}
```

Figure 4: lambda

Renvoi d'expressions lambda à partir de fonctions

Utilisation d'expressions lambda comme paramètres de fonction

Utilisation d'expressions lambda comme prédicats

Expressions Lambda avec état

Compilation d'expressions lambda stateless

```
#include <functional>                          // for std::function

std::function<void(int)> foo() {
        return [] (int x) {std::cout << x;};
}

or

void (*foo())(int) {
        return [] (int x) {std::cout << x;
}

or

auto foo() {
        return [] (int x) {std::cout << x;};
}
```

```
All 3 versions are
used the same way
;
auto l = foo();
l(10);          // Displays 10
```

Figure 5: lambda

```
foo([] (int x) {std::cout << x;});

or

auto l = [] (int x) {std::cout << x;};

foo(l);
```

Figure 6: lambda

```
void print_if(std::vector<int> nums, bool (*predicate)(int)) {
        for (int i: nums)
                if (predicate(i))
                        std::cout << i;
}

int main() {

        std::vector<int> nums {1,2,3};

        print_if(nums, [] (auto x) {return x % 2 == 0;});  // Displays evens
        print_if(nums, [] (auto x) {return x % 2 != 0;});  // Displays odds

        return 0;
}
```

Figure 7: lambda

```
[captured_variables] () -> return_type specifiers { };
```

Non-empty Capture List
Defines what information/variables should be captured

Figure 8: lambda

Lambda definition

```
auto l = [] (int x) {std::cout << x;};
```

Compiler-generated closure

```
class CompilerGeneratedName {
public:
    CompilerGeneratedName();

    void operator() (int x) {
        std::cout << x;
    }
}
```

Lambda definition

```
int y {10};

auto l = [y] (int x) {std::cout << x + y;};
```

Compiler-generated closure

```
class CompilerGeneratedName {
private:
    int y;
public:
    CompilerGeneratedName(int y) : y{y} {};

    void operator() (int x) const {
        std::cout << x + y;
    }
}
```

Capture par valeur

Utilisation de mutable pour modifier les variables capturées par valeur

Capture par référence

Capture par valeur et référence

Capture par défaut

```
int x {100};

[x] () {std::cout << x;}();     // Displays 100
```

Figure 9: lambda

```
int x {100};

[x] () mutable {
    x += 100;
    std::cout << x;     // Displays 200
}();

std::cout << x;     // Displays 100
```

Figure 10: lambda

```
int x {100};

[&x] () {x += 100;}();

std::cout << x;     // Displays 200
```

Figure 11: lambda

```
[x, y]       // Capture both x and y by value
[x, &y]      // Capture x by value and y by reference
[&x, y]      // Capture x by reference and y by value
[&x, &y]     // Capture both x and y by reference
```

Figure 12: lambda

```
[=]       // Default capture by value
[&]       // Default capture by reference
[this]    // Default capture this object by reference
```

Figure 13: lambda

Utilisation de captures par défaut et explicites

```
[=, &x]      // Default capture by value but capture x by reference
[&, y]       // Default capture by reference but capture y by value
[this, z]    // Default capture this by but capture z by value
```

Figure 14: lambda